Abstraction in Java —

Notes on **abstraction** in Java — definition, how Java implements it, examples, rules, differences, pitfalls, and quick exercises.

---

1) What is abstraction?

- Abstraction = hiding implementation details and exposing only the essential features (the *what*, not the *how*).
- Example idea: A `Car` exposes `start()` and `stop()` — you don't need to know how the engine works to use it.

---

2) Why use abstraction?

- Simplifies complex systems.
- Improves modularity & maintainability.
- Enables polymorphism (code can work with abstract types).
- Encourages separation of contract (interface) from implementation.

---

3) How Java supports abstraction (two main tools)

- **Abstract classes** (`abstract class`) — partial implementation + state.
- **Interfaces** (`interface`) — contracts (pure behavior), but since Java 8 can include default/static implementations.

---

4) Abstract class — key points & example

- Declared with `abstract` keyword.
- May contain abstract methods (no body) and concrete methods (with body).
- Can have fields (state), constructors, and any visibility (private/protected/public).
- Cannot be instantiated directly.
- A class **extends** at most one abstract class (single inheritance).
- Use when you want to provide shared code and fields to closely related classes.

Example

```
abstract class Animal {
    String name;
    public Animal(String name) { this.name = name; }
    public abstract void sound();        // abstract method
    public void sleep() {                // concrete method
        System.out.println(name + " is sleeping");
    }
}

class Dog extends Animal {
    public Dog(String name) { super(name); }
    @Override
    public void sound() {
        System.out.println(name + " says: Woof");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog("Buddy");
        a.sound();
        a.sleep();
    }
}
```

---

5) Interface — key points & example

- Declared with `interface`. Methods are abstract by default (but Java 8+ supports `default` and `static` methods; Java 9 added private methods).
- Fields (if any) are `public static final` (constants).
- A class can implement multiple interfaces (multiple inheritance of type).
- Use interfaces to define capabilities/contract (e.g., `Flyable`, `Serializable`).

Example

```
interface Flyable {
    void fly();                     // public abstract by default
    default void land() {           // default method (has body)
        System.out.println("Landing...");
    }
    static void info() {
        System.out.println("Flyable interface");
    }
}

class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println("Bird is flying");
    }
}

// Usage
Flyable.info();
Flyable b = new Bird();
b.fly();
b.land();
```

---

6) Abstract class vs Interface — quick comparison

- **State:** abstract class → can have instance fields; interface → only constants.
- **Constructors:** abstract class → yes; interface → no.
- **Method bodies:** abstract class → yes (concrete methods); interface → default & static (since Java 8), private (since Java 9).

- **Multiple inheritance:** abstract class → single `extends`; interface → multiple `implements`.
- **When to use:**
    - Use **interface** to define a role/capability and allow many implementations.
    - Use **abstract class** to share code/state between related classes.

---

7) Rules & important notes

- If a class does not implement all abstract methods from its abstract parent(s) or interfaces, it must be declared `abstract`.
- You cannot instantiate an abstract class or an interface.
- If two interfaces provide the same `default` method, the implementing class must override it to resolve ambiguity.
- Constructors in an abstract class run when a concrete subclass is instantiated (useful for initializing shared state).
- `@FunctionalInterface` marks interfaces with exactly one abstract method — usable with lambdas.

---

8) Common pitfalls

- Trying to instantiate an abstract class or interface.
- Forgetting to override all abstract methods (unless subclass is abstract).
- Assuming interfaces cannot have implementations (they can via `default`/`static`).
- Confusing “is-a” vs “can-do”: prefer abstract class for “is-a” with shared state, interface for “can-do” capabilities.

---

9) Small exercises (practice)

1. **Shapes:** Create an `abstract class Shape` with `double area()` and `double perimeter()`. Implement `Rectangle` and `Circle`.

2. **Payments:** Define `interface Payment { void pay(double amt); }` Implement `CreditCardPayment` and `UPIPayment`.

3. **Printer/Scanner:** Make `interface Printable { void print(); }` and `interface Scannable { void scan(); }`. Implement `MultiFunctionDevice` that does both. Handle default method name conflict if both interfaces had same default.

(If you want, I can give full solutions for any of these in code — say which one.)

---

10) Quick revision checklist

- Abstraction = hide details, expose essentials.
- Abstract classes = partial implementation + state.
- Interfaces = contracts, multiple implementations.
- Use `abstract` keyword for classes/methods; `implements` for interfaces.
- Remember default/static methods in interfaces (Java 8+).

🔹 Abstract Class in Java

An **abstract class** is a class that cannot be instantiated (cannot create objects directly). It is used to achieve **partial abstraction**.

Features:

1. Declared using the `abstract` keyword.

2. Can contain **abstract methods** (without body) and **non-abstract methods** (with body).

3. Can have **constructors** (but cannot be instantiated directly).

4. Can have **instance variables** as well as **static variables**.

5. Can define **final methods** (cannot be overridden).
6. Supports **method overloading** and **method overriding**.
7. Can extend only **one abstract class** (since Java doesn't support multiple inheritance with classes).
8. Access modifiers (`public`, `protected`, `private`) are allowed for methods and variables.
9. Useful when classes are related by IS-A relationship and you want to provide **common base functionality**.

🔹 Interface in Java

An **interface** is a collection of **abstract methods** and **static/final variables**. It is used to achieve **100% abstraction** (before Java 8) and **multiple inheritance** in Java.

Features:

1. Declared using the `interface` keyword.
2. Cannot have constructors.
3. All variables inside an interface are by default `public static final`.
4. All methods are **public abstract** by default (till Java 7).
5. From **Java 8 onwards**:
    - Interfaces can have **default methods** (methods with body using `default` keyword).
    - Interfaces can have **static methods**.
6. From **Java 9 onwards**:
    - Interfaces can have **private methods** (to share common code between default methods).
7. A class can **implement multiple interfaces** (solves the multiple inheritance problem).

8. An interface can **extend another interface** (but cannot extend a class).
9. Provides a contract — "what a class must do" but not "how to do it".

🔹 Key Differences Between Abstract Class and Interface

Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Can have abstract + concrete methods	Before Java 8 → only abstract methods, after Java 8 → default & static methods too
Variables	Instance variables allowed	Only `public static final` (constants)
Constructors	Allowed	Not allowed
Inheritance	A class can extend only 1 abstract class	A class can implement multiple interfaces
Access Modifiers	Methods can be `public`, `protected`, `private`	Methods are always `public` (except private helper methods from Java 9)
Use Case	When classes share common state + behavior	When multiple unrelated classes must follow a contract

---

👉 In short:

- Use **abstract class** when you want to provide **base implementation** along with abstraction.
- Use **interface** when you want to provide **contract (rules)** that multiple classes must follow, supporting multiple inheritance.
